

1장

딥러닝과 파이토치 라이브러리 소개

1장에서 다루는 내용

- 딥러닝이 바꾼 머신러닝 접근 방식
- 파이토치가 딥러닝에 최적인 이유
- 대표 딥러닝 프로젝트 살펴보기
- 예제 실습에 필요한 하드웨어 사양

인공지능artificial intelligence의 정의는 다소 모호하여 엄청난 양의 연구 조사를 통한 지식체계라는 의미뿐 아니라 다소 혼란하고 비현실적인 기대감과 때때로 공상과학에서 조장하는 공포감까지 담은 용어가 돼버렸다. 실제 인공지능을 바라보는 현실은 낙관적이더라도 누군가는 기계가 사람처럼 말을 알아듣고 학습한다고 주장할지도 모른다. 허나 그런 사람은 잘 모르고 하는 얘기일 가능성이 크다. 이 책에서 다루는 일반화된 알고리즘 분류는 기존에 사람만 할 수 있던 일을 자동화해주며 복잡한 비선형 프로세스들을 효과적이면서 꽤 그럴듯하게 처리한다.

예를 들어 <https://talktotransformer.com>에서 시연 가능한 GPT-2 언어 모델은 주어진 텍스트를 기반으로 단락의 나머지를 완성한다. 여러분이 읽고 있는 이 문단을 그대로 입력하면 나머지를 다음과 같이 완성해준다.¹

다음으로 우리는 이메일 주소 뭉치로 만들어진 여러 구절 리스트를 입력한 후, 프로그램이 리스트를 문장으로 파싱할 수 있는지 확인하려 한다. 다시 말하지만 이 작업은 이 글의 시작 부분에서 검색하는 작업보다 훨씬 섬세하고 복잡하다. 그렇다고 해도 이 작업은 여러분이 여러 프로그래밍 언어로 문장 구조를 만들어내는 것에 대한 기본을 이해할 수 있도록 도와준다.

황설수설하고 불명확한 글이지만 기계치고는 놀라울 정도로 논리정연하다.

예전에는 사람만 가능했던 이런 작업을 일일이 손으로 작성한 규칙이 아니라 여러 예를 보고 학습으로 수행했다는 점이 더욱 인상적이다. 어떤 면에서, 우리는 그간 지능은 자의식을 포함한 개념이라 생각해왔지만 이런 결과를 보면 작업을 수행하는 데 자의식은 굳이 필요하지 않다는 점을 알게 된다. 궁극적으로는 컴퓨터가 지능을 가지는지에 대해 질문할 필요도 없을 것 같다. 에츠허르 W. 데이크스트라²는 “머신이 생각할 수 있는가”라는 질문에 대해 그건 마치 잠수함이 수영할 수 있는지 묻는 것과 같다고 답했다.

이 책에서 우리가 말하는 일반화된 알고리즘 분류는 AI의 하위 범주인 딥러닝^{deep learning}에 해당한다. 딥러닝은 가르치기에 적합한 예제들을 사용하여 심층 신경망^{deep neural network}이라는 수학적 개체를 훈련시키는 일을 한다. 딥러닝은 대량의 데이터를 사용해 입력과 출력이 동떨어진 복잡한 함수들을 근사하는 방법을 사용한다. 관계 파악이 어려운 입력과 출력이란, 입력받은 이미지를 설명하는 문장을 출력하는 사례, 입력받은 대본을 사람이 읽는 것처럼 자연스럽게 낭송하는 사례, 혹은 깃발이 함께 있는 골든 리트리버 이미지를 추론해서 “네, 골든 리트리버가 여기 있네요.”라고 응답하는 사례들을 들 수 있다. 딥러닝의 이런 능력 덕분에 우리는 최근까지 사람만이 독점적으로 행할 수 있던 기능을 프로그램에게 부여할 수 있게 됐다.

1.1 딥러닝 혁명

이와 같은 딥러닝 접근 방식이 가져온 패러다임 변화를 이해하려면 이전 상황이 어떠했는지를 먼저 이해해야 한다. 십여 년 전까지 머신러닝^{machine learning}이라고 분류되는 광범위한 시스템은 피처 엔지니어링^{feature engineering}에 크게 의존했다. 피처 엔지니어링이란 다운스트림 알고리즘(일례로 분류기)이 새로운 데이터를 기반으로 올바른 결과물을 낼 수 있도록 촉진하기 위해 입력 데이터를 변환하는 작업을 말한다. 피처 엔지니어링은 다운스트림 알고리즘이 문제를 풀 수 있도록 올바른 변환 규칙들을 포함한다. 예를 들어 손글씨 숫자에서 0과 1의 이미지를 구분해내려면, 이미지의 테두리선 분포를 추정하기 위한 필터 집합을 구성해야 하며, 이를 활용해 주어진 테두리선의 방향 분포로 올바른 숫자를 예측하게끔 분류기를 훈련시켜야 한다. 추가로 0이나 8 혹은 특이하게 쓴 숫자 2로 만들어진 동그라미가 이미지에 몇 개 있는지를 피처로 사용할 수도 있다.

반면 딥러닝은 이와 같은 숫자 인식 작업을 성공적으로 수행하기 위해 원본 데이터로부터 자동으로 (숫자의) 표현들을 찾아낸다. 0과 1을 구분하기 위해 필터는 데이터와 대응 레이블을 반복적으로 훈련시킨 결과, 자동으로 개선된다. 그렇다고 딥러닝 기술이 부상한다고 해서 피쳐 엔지니어링이 사라지는 건 아니다. 학습 시스템에는 어떤 형태로든 사전 지식을 주입할 필요가 있기 때문이다. 하지만 일단 데이터를 넣으면 주어진 예제로부터 유용한 표현을 추출하는 신경망의 능력은 딥러닝을 좀 더 강력하게 만든다. 때문에 딥러닝 실무에서는 표현을 수작업으로 만드는 것보다 신경망 내부의 수학적 개체를 조작해 훈련 데이터로부터 표현을 자동으로 훈련시키는 데 집중한다. 그리고 꽤 많은 경우, 수작업의 결과에 비해서도 자동으로 생성된 피처가 훨씬 더 뛰어나다! 혁신적인 여느 기술과 마찬가지로 이런 사실은 관점의 변화를 이끌어내고 있다.

그림 1.1의 왼쪽 그림에서는 엔지니어링 피처를 정의하고 학습 알고리즘에 넣느라 바쁜 모습을 보여준다. 실무자가 피처를 잘 만들어낼수록 학습도 좋은 결과를 가져다 준다. 반면 오른쪽 딥러닝 그림에서는 원본 데이터가 알고리즘에 바로 들어가고 작업 절차에서 성능 최적화의 유도 과정에 따라 저수준에서 고수준의 피처들이 자동으로 추출된다. 이 방식은 사람이 개입해 직접 알고리즘을 학습시키는 것과 유사한 수준으로 좋은 결과를 가져다 준다.

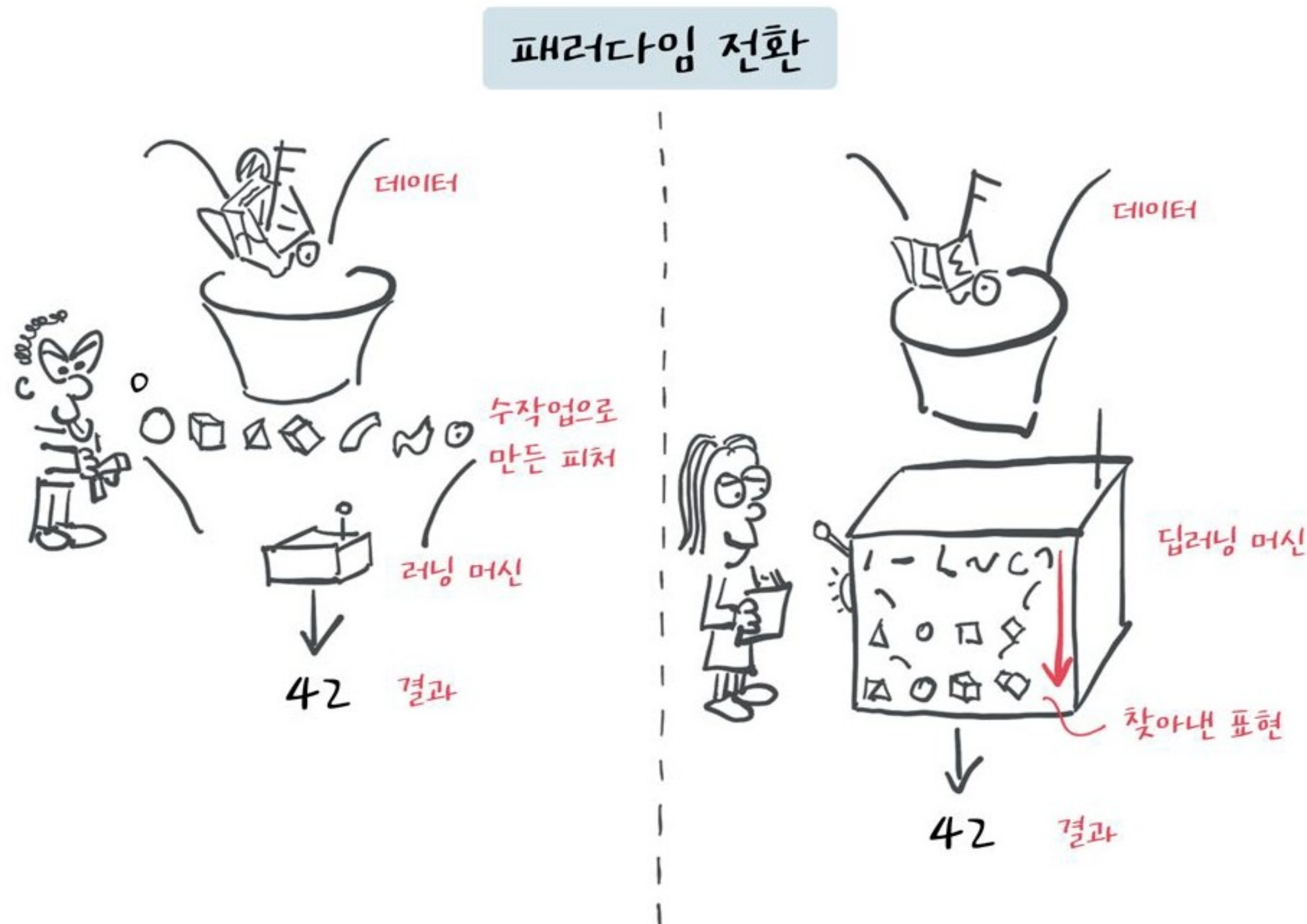


그림 1.1 딥러닝은 데이터와 연산량 증가를 통해 피처에 관련된 수작업을 필요 없게 만들었다.

패러다임 전환

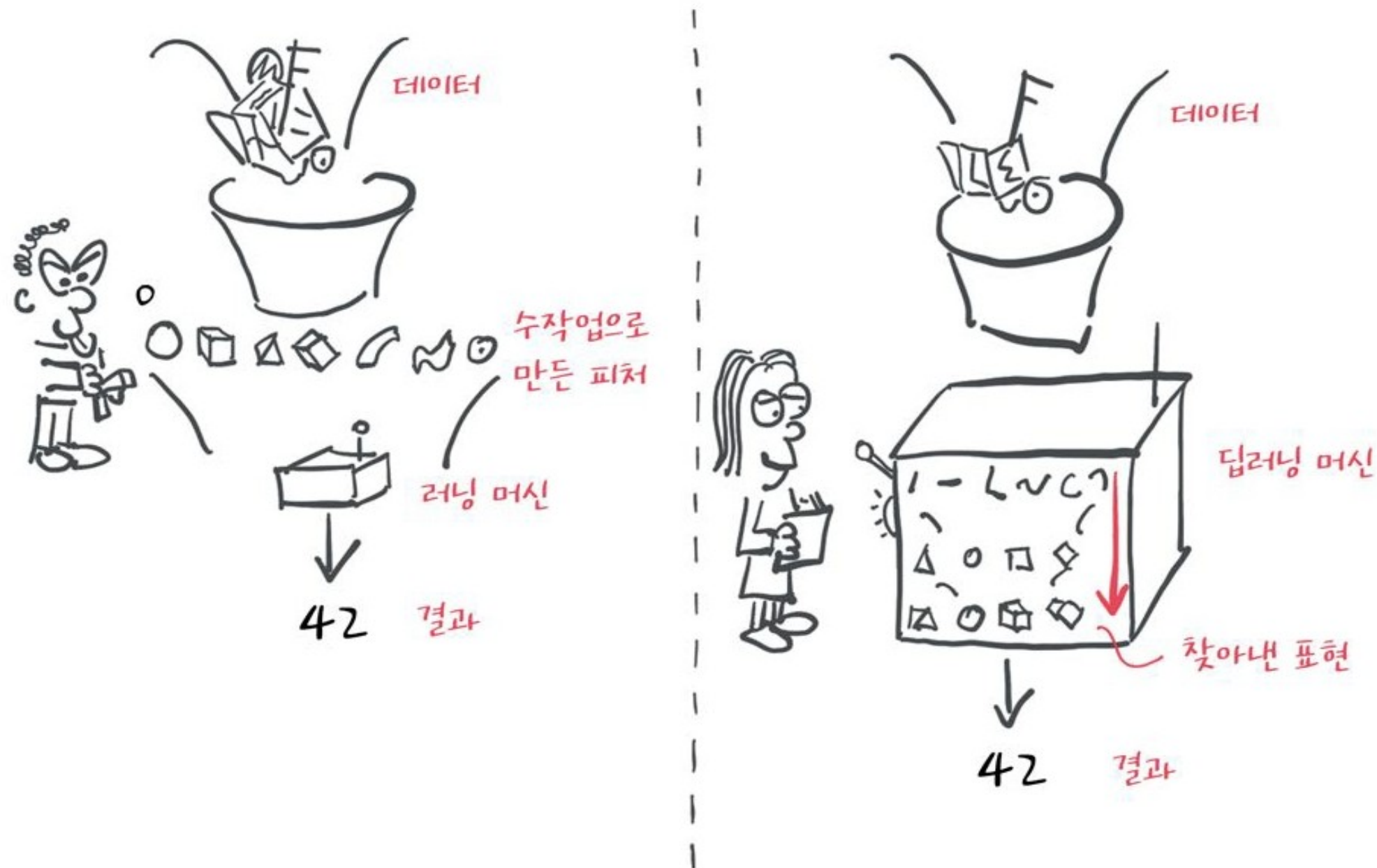


그림 1.1 딥러닝은 데이터와 연산량 증가를 통해 피처에 관련된 수작업을 필요 없게 만들었다.

그림 1.1의 오른쪽 이미지를 잘 살펴보면 딥러닝을 성공적으로 실행하기 위해 필요한 요구사항을 가늠해볼 수 있다.

- 가지고 있는 데이터가 무엇이든 일단 넣을 수 있어야 한다.
- 딥러닝 머신을 정의해야 한다.
- 유용한 표현을 얻고 머신이 원하는 출력 결과를 낼 수 있도록 자동화된 훈련 방법을 만들어야 한다.

이쯤 되면 이 책에서 계속 얘기하는 훈련^{training}은 구체적으로 무엇인지 궁금할 테다. 훈련 과정에서는 기대한 값과 실제 모델 출력의 차이를 점수로 제공하기 위해 참조 데이터 그리고 모델의 출력값을 사용한 실수 함수로 기준^{criterion}을 만들어 사용한다. 통상적으로 점수는 낮을수록 좋다. 훈련은 기준을 통해 점점 더 낮은 점수를 얻도록 점진적으로 딥러닝 머신을 변경하여, 훈련 중에 입력되지 않은 데이터에 대해서도 낮은 점수를 달성하게 만든다.

1.2 딥러닝을 위한 파이토치

파이토치^{PyTorch}는 딥러닝 프로젝트를 위한 파이썬 프로그램 라이브러리다. 파이썬 언어를 사용하듯이 딥러닝 모델을 표현할 수 있어 매우 유연하다. 연구 단체의 얼리어답터들은 파이토치의 접근성과 사용 편의성을 릴리스 첫 해부터 알아차렸고 이후 파이토치는 다양한 딥러닝 애플리케이션 중에서도 가장 각광받는 딥러닝 도구로 자리잡았다.

처음 프로그래밍을 배우려는 사람에게 파이썬이 안성맞춤이듯이 딥러닝을 도입하는 데는 파이토치 만한 것이 없다. 파이토치는 현실에서 고수준의 작업을 위한 전문적인 도구로도 전혀 손색이 없음이 검증됐다. 파이토치의 명확한 문법과 현대적인 API, 그리고 쉬운 디버깅은 딥러닝 도입을 위한 최선의 선택이다. 여러분이 처음으로 딥러닝 라이브러리를 공부한다면 파이토치를 선택하기를 강력히 추천한다. 물론 파이토치 외에 딥러닝 라이브러리를 추가로 더 배워야 할지 여부는 여러분의 선택에 달려 있다.

핵심으로 돌아가면 그림 1.1의 딥러닝 머신은 입력을 출력으로 매핑하는 다소 복잡한 수학 함수라고 생각할 수 있다. 이 함수를 만들기 위해 파이토치는 텐서^{tensor}라는 핵심 데이터 구조를 제공한다. 텐서는 넘파이^{NumPy} 배열과 여러 면에서 유사한 다차원 배열이다. 텐서라는 자료구조 위에서 파이토치는 전용 하드웨어를 십분 활용해 수학 연산을 빠르게 수행한다. 그 덕분에 파이토치는 신경망 아키텍처를 설계하고 개별 혹은 분산 컴퓨팅 자원에서 훈련하기 매우 편리하다는 장점을 제공한다.

이 책은 파이썬에 이미 익숙한 소프트웨어 엔지니어나 데이터 과학자 혹은 호기심 많은 학생들이 딥러닝 프로젝트를 만드는 출발점으로 삼을 수 있도록 쓰여졌다. 가능한 한 많은 사람이 읽고 도움이 되기를 바라며 이 책의 개념을 이해해서 다른 분야에도 적용해보기를 바란다. 이런 목적을 달성하기 위해 우리 저자진은 직접 따라해볼 수 있는 예제를 준비했고 컴퓨터가 준비됐다면 예제를 돌려보고 더 알아볼 수 있게 했다. 책을 다 읽고 나서는 꽤 잘 만들어진 공식 문서의 도움도 받아가면서 여러분이 직접 데이터를 마련하고 딥러닝 프로젝트를 만들어 보기 바란다.

이 책에서는 파이토치로 딥러닝 시스템을 만드는 실용적인 면을 강조했지만, 우리는 기본적인 딥러닝 도구를 실제로 다뤄볼 수 있도록 소개하는 것이 새로운 기술을 습득하게 하는 것 이상의 의미가 있다고 생각한다. 이 책은 앞으로 수십 년 동안 많은 소프트웨어 프로젝트의 근간이 될 실무 지식을 가르쳐서 다양한 분야에 속한 새로운 세대의 과학자와 엔지니어 그리고 전문가들이 준비를 갖추 수 있게 만드는 디딤돌을 마련하고자 한다.

이 책을 최대한 활용하려면 다음과 같은 두 가지 사항이 필요하다.

- 파이썬 프로그래밍 경험. 이 책에서는 파이썬을 따로 친절하게 다루지 않는다. 파이썬의 데이터 타입과 클래스, 부동소수 같은 내용은 미리 숙지해둬야 한다.
- 직접 만들어보며 체득할 의지. 기초부터 시작해서 실무 지식을 하나씩 쌓아갈 것이므로 잘 따라만 온다면 훨씬 배우기 쉬울 것이다.

이 책은 3개 부로 구성된다. 1부에서는 기본 개념을 설명하며, 파이토치가 제공하는 기능들을 상세하게 살펴면서 그림 1.1에서 표현한 딥러닝을 실제 코드로 만들어볼 것이다. 2부는 의학 영상 자료를 가지고 처음부터 끝까지 만드는 프로젝트로서, 1부에서 소개한 기본 개념을 토대로 CT(컴퓨터 단층 촬영) 사진에서 종양을 찾아 식별하는 예제를 제시하며, 좀 더 수준 높은 주제도 다룬다. 3부에서는 딥러닝 모델의 실제 적용을 위해 파이토치가 제공하는 배포 기능을 간단히 소개하며 마무리한다.

딥러닝은 광범위하며, 이 책에서 우리가 다루는 부분은 극히 일부일 뿐이다. 아마도 이 책에서는 파이토치를 사용해 2차원과 3차원 데이터셋 이미지를 처리해서 소규모로 분류하고 분할하는 예제가 제일 흥미로울 것이다. 이 책은 파이토치의 실용적인 면에 초점을 둬으로써, 독자 여러분이 비전 인식 같은 실세계의 머신러닝 문제를 풀거나 학계에서 떠오르는 새로운 모델을 탐구하게 도와 줄 것이다. 주요 딥러닝 연구에 대한 최신 논문은 출간 전 공개 저장소인 ‘아카이브 arXiv’(<https://arxiv.org>)³에서 볼 수 있다.

1.3 왜 파이토치인가?

이미 언급했듯이 딥러닝은 우리가 다양한 분야의 복잡한 작업을 수행할 수 있게 해준다. 예를 들어 기계 번역이나 전략 게임 실행 혹은 어수선한 장면에서의 물체 인식 등의 작업은 모두 우리가 만든 모델을 여러 예제 데이터에 노출해 학습시켜줘야 가능해진다. 실제 학습을 위해서는 다양한 문제에 적용할 수 있을 정도로 유연하면서도, 적당한 시간 안에 많은 양의 데이터로 훈련 가능한 효율적인 도구가 필요하다. 또 이 도구로 훈련된 모델은 다양한 입력에 대해 정확하게 동작해야 한다. 이런 점을 염두에 두고 우리가 파이토치를 사용하기로 결정한 몇 가지 이유를 살펴보자.

파이토치를 추천하는 이유 중 하나는 단순함이다. 많은 연구원과 실무자의 얘기를 들어 보면 파이토치는 학습이나 활용, 확장, 디버깅 면에서 모두 수월하다고 한다. 주의 사항이나 추천 용례가 있는 정교한 다른 라이브러리처럼 파이토치는 파이썬스러운 면이 있어서 파이썬을 써본 개발자라면 친숙하게 사용할 수 있다.

더 구체적으로 말하자면, 파이토치로 딥러닝 머신을 프로그래밍하는 것은 정말 자연스럽다. 숫자나 벡터 혹은 행렬이나 배열을 담을 수 있는 데이터 타입인 Tensor를 기본으로 제공한다. 부가적으로 텐서 연산을 위한 함수도 지원한다. 이런 것들을 사용해 프로그램을 점차 확장해갈 수 있고 원한다면 파이썬처럼 인터랙티브하게 쓸 수도 있다. 넘파이를 잘 안다면 금방 친숙해질 것이다.

특히 파이토치가 딥러닝을 위해 주요하게 제공하는 두 가지 특별한 기능이 있다. 하나는 그래픽 연산 유닛인 GPU로 연산을 가속하는 것인데 CPU에서 처리하는 경우보다 50배나 빠른 결과를 보여주기도 한다. 다른 하나는 훈련을 위해 딥러닝이 사용하는 일반 수학식에 대해 산술 최적화를 지원하는 점이다. 이런 기능은 딥러닝뿐만 아니라 과학 계산에서도 유용하다. 이렇게 파이토치는 파이썬에서의 과학 계산 최적화를 지원하는 고성능 라이브러리라는 특징을 지닌다.

파이토치는 풍부한 표현력을 염두에 두고 설계되었기에, 개발자는 라이브러리를 사용할 때 불필요한 복잡함 없이 정교하게 모델을 구현할 수 있다(파이토치는 프레임워크가 아니다!). 단언컨대 파이토치는 딥러닝 영역의 아이디어를 파이썬 코드로 즉시 매끈하게 만들어낼 수 있는 구현 도구라 할 수 있다. 이런 이유로 파이토치는 연구 분야에서 광범위하게 채택됐으며 국제 학회에서도 높은 빈도로 언급된다.⁴

파이토치는 연구 개발 산출물을 제품화하는 경우에서도 설득력 있는 사례를 제공한다. 처음에는 연구 작업 흐름에 초점을 맞췄지만, 파이썬에 의존하지 않고서도 추론 가능한 모델을 배포하기 위해 고성능 C++ 런타임을 갖추기 시작하면서 파이토치는 C++로 모델을 설계하고 훈련시키는 과정에서도 사용될 수 있다. 또한 파이토치는 다른 언어를 지원하는 바인딩과 모바일 장비에 배포하기 위한 인터페이스도 늘리고 있다. 이런 기능 덕분에 사람들은 파이토치의 유연함을 심분 활용할 수 있었고 파이썬 런타임을 사용하기 어렵거나 비용이 드는 경우에도 애플리케이션 적용이 가능해졌다.

파이토치의 사용 편의성과 고성능을 자랑하는 주장이 다소 시시해보일 수도 있겠지만 여러분이 이 책의 중반을 넘어서게 되면 우리의 주장이 다 근거 있다는 사실을 수긍할 수밖에 없을 것이다.

1.3.1 딥러닝 경쟁 구도

다소 과장된 비유이긴 하지만 2017년 1월 파이토치 1.0 릴리스는 마치 다양한 종류의 동물 화석이 갑작스럽게 출현했던 지질학적 사건인 캄브리아기 대폭발처럼, 딥러닝 라이브러리, 래퍼 wrapper, 데이터 교환 포맷 등이 쏟아져 나오던 시기를 합병과 대통합의 시기로 바꿔놓았다.

노트

딥러닝 분야는 너무 빠르게 움직이기 때문에 아마 출간본으로 이 내용을 읽을 무렵에는 시대에 뒤떨어진 이야기처럼 들릴 가능성이 높다. 낯선 라이브러리들이 언급되더라도 이해하기 바란다.

파이토치 첫 베타 버전이 릴리스되던 당시의 동향은 다음과 같았다.

- 테아노^{Theano}와 텐서플로^{TensorFlow}는 초기의 저수준 라이브러리였고, 사용자가 먼저 연산 그래프를 정의한 후 실행하도록 만들어진 모델이었다.
- 라자냐^{Lasagne}와 케라스^{Keras}는 테아노를 고수준으로 래핑한 것으로, 케라스는 텐서플로와 CNTK도 래핑했다.
- 카페^{Caffe}와 체이너^{Chainer}, 다이넷^{DyNet}, 토치^{Torch}(루아^{Lua} 기반으로 만들어진 파이토치의 원형), 엠엑스넷^{MXNet}, CNTK, DL4J 등이 이 바닥의 많은 빈틈을 메우고 있었다.

그 후로 대략 2년 동안 경쟁구도에 엄청나게 많은 변화가 있었다. 커뮤니티는 특수한 경우가 아니면 파이토치나 텐서플로의 양 진영으로 나뉘었고, 틈새를 노리는 특정 분야를 제외하고는 다른 라이브러리 채택은 점차 줄어들고 있었다.

- 거의 최초의 딥러닝 프레임워크인 테아노는 적극적인 개발이 중단됐다.
- 텐서플로 동향
 - 케라스를 완전히 흡수해서 최상위 클래스 API로 붙여놓음
 - ‘이거^{eager} 모드’로 불리는 즉시 실행을 제공해 파이토치와 유사한 계산 방식을 지원함
텐서플로 2.0 릴리스에서는 이거 모드가 기본이 됨
- 텐서플로에서 독립하여 개발된 구글의 JAX 라이브러리는 GPU를 사용한 넘파이 연산과 성능이 비슷하고 자동미분^{autograd}과 JIT 기능으로 큰 인기를 끌게 됨
- 파이토치 동향
 - 백엔드에 카페2가 자리함
 - 재사용하던 루아 기반의 토치 저수준 코드를 다 바꿈
 - 벤더 중립적인 모델을 만들어 포맷을 교환할 수 있게 해주는 ONNX를 지원함
 - 토치스크립트^{TorchScript}라는 ‘그래프 모드’ 런타임으로 지연 실행 기능을 추가함
 - 1.0 버전을 릴리스함
 - 후원 기업으로부터 CNTK와 체이너를 대체할 프레임워크로 선택받음

텐서플로는 제품화를 위한 견고한 파이프라인과 함께 업계 전반에 걸쳐 두터운 커뮤니티와 높은 인지도를 확보했다. 파이토치는 사용하기 쉬운 장점을 살려 연구와 교육 커뮤니티에 침투했고 연구원과 졸업생들이 학생들을 교육하고 업계로 진출하면서 동력을 얻기 시작했다. 제품 수준의 솔루션 관점에서도 박차를 가했다. 한 가지 흥미로운 점은 토치스크립트나 이거 모드 같은 기능이 등장하면서 파이토치와 텐서플로가 각자만의 기능을 서로 흡수해온 것인데, 실제로는 상세 기능이나 사용 방법에 꽤 많은 차이가 존재한다.

1.4 파이토치 딥러닝 프로젝트 둘러보기

앞에서 몇 가지 파이토치 구성 요소를 대략 살펴보았으니 이번에는 주요 요소를 알아보고 머릿속에 지도를 만들어 보자. 딥러닝 프로젝트를 수행할 때 파이토치에 요구하는 사항이 무엇인지를 생각해보면 쉽게 파악할 수 있다.

파이토치의 ‘파이Py’는 파이썬에서 따왔지만 실제로는 파이썬이 아닌 코드가 많이 들어 있다. 성능상의 이유로 대다수 파이토치의 코드는 C++로 컴파일됐으며, 대규모 병렬 프로그램을 GPU에서 돌릴 수 있는 C++ 언어스러운 CUDA(www.geforce.com/hardware/technology/cuda)를 사용했다. 이 때문에 C++로 직접 파이토치를 구동하는 방법도 있는데 이에 대해서는 15장에서 다룰 것이다. 이는 모델을 제품화하기 위해 안정적인 전략을 제공한다. 우리 책에서는 대부분 파이썬으로 파이토치를 다루고 모델을 만들어 훈련시키며 실제 문제도 훈련한 모델을 사용해 풀어볼 예정이다.

파이토치의 놀라운 활용성과 파이썬 생태계와의 폭넓은 통합 이면에는 파이썬 API가 있다. 일단 파이토치가 무엇인지에 대해 떠올려 볼 수 있을 정도만 살펴보자.

앞서 언급한 바와 같이 파이토치는 기본적으로 다차원 배열 혹은 파이토치 언어로는 텐서라 부르는 자료구조를 사용하는 라이브러리다(3장에서 자세히 다룬다). 따라서 텐서를 사용하는 다양한 연산 라이브러리를 torch 모듈로 제공한다. 텐서 자료구조나 이를 사용한 연산은 CPU나 GPU에서 수행한다. CPU에서 수행하던 연산을 GPU로 옮기는 과정은 파이토치에서 단지 한두 개의 함수 호출로 가능하다. 파이토치가 제공하는 두 번째 핵심 기능으로, 특정 텐서에서 수행한 모든 연산을 기억해뒀다가 주어진 입력값을 기준으로 미분값을 자동 계산해준다. 파이토치의 계산 최적화 기능은 내부적으로 파이토치의 자동미분 엔진이 텐서를 사용하기 때문에 가능하다.

텐서와 자동미분을 제공하는 텐서 표준 라이브러리로 파이토치는 물리 연산과 렌더링, 최적화, 시뮬레이션, 모델링 등 많은 곳에 사용되며 앞으로도 다양한 응용과학 분야에서 참신하게 활용될 것이다. 파이토치는 신경망을 만들고 훈련하는 데 필요한 모든 빌딩 블록을 제공하기 위해 처음 등장한 딥러닝 라이브러리다. 그림 1.2에서 데이터를 읽고 모델을 훈련한 후 만들어진 모델을 제품으로 배포하는 데 필요한 일반적인 과정을 볼 수 있다.

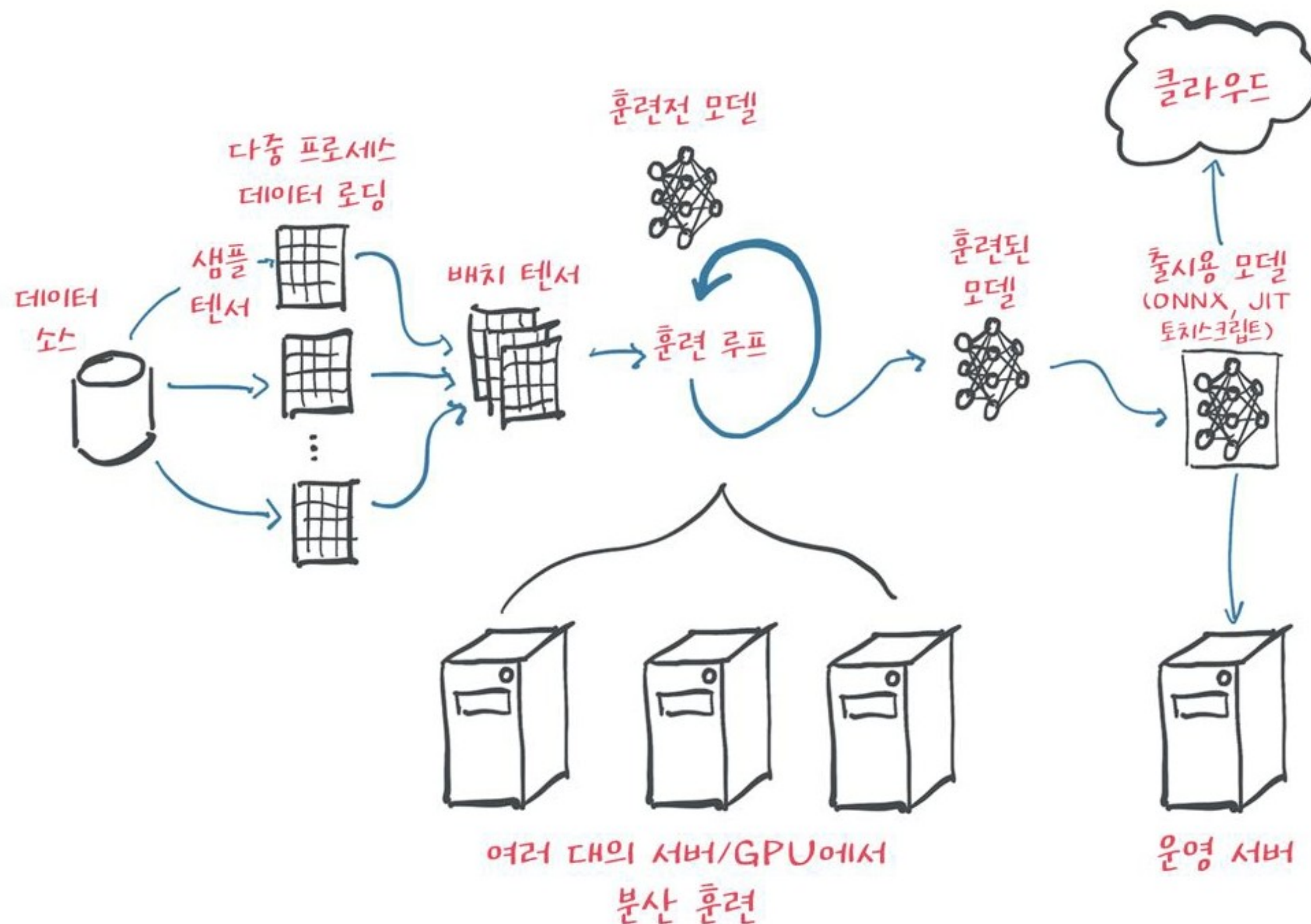


그림 1.2 데이터 로딩, 훈련, 제품화를 위한 배포까지 포괄한 파이토치 프로젝트의 기본 구조

신경망 구축을 지원하는 파이토치 핵심 모듈은 `torch.nn`에 있으며 공통적인 신경망과 아키텍처적인 구성 요소를 제공한다. 여기에는 완전 연결 계층^{fully connected layer}이나 컨볼루션층, 활성화 함수, 손실 함수가 모두 포함된다(책의 나머지 부분에서 이 모든 내용이 의미하는 바에 대해 자세히 살펴볼 것이다). 이 요소들은 그림 1.2의 중앙에 보이는 훈련전 모델^{untrained model}을 만들고 초기화하는 데 필요하다. 모델 훈련을 위해서는 몇 가지 추가적인 사항이 더 필요하다. 훈련 데이터를 얻을 수 있는 곳, 모델을 훈련 데이터에 맞춰주는 옵티마이저^{optimizer}, 그리고 모델과 데이터로 모델 훈련을 위해 필요한 계산을 수행할 하드웨어 등이 있어야 한다.

그림 1.2의 왼쪽부터 보면, 훈련 데이터가 모델에 이르기까지 많은 데이터 처리가 필요함을 알 수 있다.⁵ 가장 먼저 저장소를 비롯한 데이터 출처로부터 물리적으로 데이터를 가져오는 작업이 필요하다. 그리고 데이터에서 가져온 각각의 샘플을 파이토치가 다룰 수 있는 텐서로 변환해야 한다. 입수한 데이터를 표준화된 파이토치 텐서로 전환하려면 `torch.utils.data`에 있는 `Dataset` 클래스가 필요하다. 데이터 변환은 프로젝트마다 다르기 때문에 데이터를 가져오기 위한 부분은 직접 구현해야만 할 것이다. 다양한 타입의 데이터를 원하는 형태의 텐서로 표현하는 방법은 4장에서 설명한다.

특히 접근 대기시간으로 인해 데이터 저장소는 대부분 느리기 때문에 병렬로 로딩해야 한다. 한편 그동안 파이썬은 느린 저장소를 위한 쉽고 효율적인 병렬 처리 없이도 사랑받아 왔지만, 우리는 이제 여러 개의 훈련 샘플을 묶어놓은 텐서 형태인 배치^{batch}로 데이터를 묶기 위해 병렬 처리가 필요하다. 정교함이 요구되는 작업이지만 다행히 파이토치에서는 이를 일반화해 DataLoader 클래스에 멋지게 담아냈다. DataLoader 클래스를 잘 사용하면 알아서 자식 프로세스가 뜨고 백그라운드로 동작하며 데이터셋으로부터 데이터를 읽어 훈련 루프가 필요할 경우 바로 전달하도록 준비할 수 있다. Dataset과 DataLoader는 7장에서 소개한다.

데이터를 배치 샘플 묶음으로 만들고 나면 그림 1.2 중앙부에 그려진 훈련 루프로 넘어갈 수 있다. 대부분 훈련 루프는 파이썬의 for 문으로 작성한다. 간단한 경우라면 모델은 필요한 계산을 로컬 CPU나 한 개의 GPU에서 수행하며 루프를 돌면서 데이터가 준비되면 바로 계산을 수행할 수 있다. 이 책에서는 이 부분을 여러분이 직접 준비하도록 안내한다.

훈련 루프는 매번 데이터 로더에서 얻은 샘플로 모델을 평가한다. 모델의 출력을 기대값(목표 값)과 비교하는데 이때 **기준**^{criterion} 혹은 **손실 함수**^{loss function}를 사용한다. 파이토치는 모델을 만들 때 필요한 요소뿐만 아니라 마음대로 사용 가능한 손실 함수도 제공한다. 손실 함수 역시 torch.nn에 있다. 손실 함수로 출력 결과를 이상적인 결과와 비교한 후 다음 출력이 목표 출력을 더 잘 묘사하게끔 모델을 조정해야 한다. 앞에서 언급한 파이토치 자동 미분 엔진을 여기에서 사용한다. 조정을 수행하는 **옵티마이저**도 필요한데 이는 torch.optim에서 제공한다. 훈련 루프와 옵티마이저는 5장에서 다루며, 6~8장에 해당하는 2부에서 조금 큰 프로젝트를 시작하며 본격적으로 사용해본다.

최근에는 그림 1.2의 하단처럼 더 큰 모델을 훈련시킬 목적으로 자원을 제공하는 여러 GPU나 여러 컴퓨터와 같은 복잡한 하드웨어를 점점 더 많이 사용하고 있다. 복잡한 하드웨어 사용을 위해 `torch.nn.parallel.DistributedDataParallel`과 `torch.distributed` 서브 모듈이 쓰인다.

훈련 루프가 너무나 재미없어 보일지 모르지만 딥러닝 프로젝트에서 가장 많은 시간을 소비하는 부분이다. 이 과정이 끝나면 모델이 원하는 작업에 맞게 최적화되는 보상을 얻는다. 그림의 훈련 루프 오른쪽에는 **훈련된 모델**^{trained model}이 그려져 있다. 모델이 원하는 작업을 수행하도록 만드는 것도 대단하지만 이 모델이 유용하려면 실제 작업이 필요한 곳에서 활용돼야 한다. 그림 1.2의 오른쪽에 그려진 **배포**^{deployment} 프로세스는 모델을 서버에 올리거나 클라우드 엔진에서 로딩할 수 있도록 내보내는 과정이다. 이 지점에서 더 큰 애플리케이션의 일부로 통합되거나 휴대폰에서 동작시키기도 한다.

배포 과정 중 눈여겨볼 만한 것은 모델 내보내기다. 이미 언급했듯이 파이토치는 기본이 즉시 실행 모드(이거 모드)다. 파이토치와 관련된 명령어가 파이썬 인터프리터에 전달되면 각 연산은 즉시 기반 C++나 CUDA 구현을 통해 수행된다. 텐서를 대상으로 하는 연산이 많을수록 백엔드 구현은 더 많이 연산을 수행한다.

파이토치는 **토치스크립트**TorchScript를 통해 모델을 미리 컴파일하는 방식도 제공한다. 토치스크립트를 사용하면 파이토치는 모델을 직렬화해서 여러 명령어 집합으로 바꾸고 나중에 파이썬에 의존하지 않고서 C++ 프로그램이나 모바일 기기에서 실행할 수 있게 된다. 텐서 연산과 연관된 제한된 명령어 집합을 지원하는 가상 머신으로 생각하면 된다. 이 기능은 모델을 파이썬 런타임에서 사용하는 토치스크립트나 ONNX라 부르는 표준화된 포맷으로 내보내어 사용할 수 있으며, 파이토치의 실서비스 배포를 위한 기본 기능이다. 이 내용은 15장에서 다룰 예정이다.

1.5 하드웨어와 소프트웨어 요구사항

이 책의 내용을 이해하려면 직접 코드를 작성하며 큰 수 행렬 곱셈 같은 덩치 큰 숫자 연산을 실행해봐야 한다. 이제 곧 밝혀지겠지만, 사전에 훈련된 신경망을 신규 데이터로 돌려보는 것은 최신 노트북이나 PC에서도 무리 없이 가능하다. 심지어 사전에 훈련된 신경망을 가져다 특화시키기 위해 일부분만 신규 데이터로 재훈련시키는 작업을 할 때는 굳이 고성능 머신이 아니어도 괜찮다. 1부에서 다루는 모든 내용은 일반 PC나 노트북만으로도 충분히 따라할 수 있다.

하지만 2부에서 다루는 고급 예제의 전체 훈련을 완료하려면 CUDA를 지원하는 GPU가 필요하다. 2부에서 사용하는 기본 파라미터는 8기가 램 GPU(NVIDIA GTX 1070 이상 급)을 사용하는 데 가용 메모리가 적을 경우 파라미터를 조정하면 된다. 여기서 분명히 짚고 넘어가겠다. 인내심이 충분하다면 고사양 하드웨어는 필요 없겠지만 GPU에서의 훈련은 실행 시간을 지수적으로 줄여준다(통상 40배에서 50배 정도 빠르다). 개별적으로 보면, 한 번의 파라미터 조정 연산은 오늘날의 노트북 CPU로도 충분히 빠르다(1초 미만에서 수 초 사이). 문제는 훈련 오차를 최소화 하는 과정에서 신경망 파라미터를 지속적으로 반복해서 엄청나게 많이 조정해야 하는 점이다.

조금 큰 규모의 신경망을 대규모의 실세계 데이터셋으로 처음부터 훈련시키는 경우라면 좋은 GPU를 장착한 워크스테이션에서도 몇 시간에서 며칠이 걸릴 수 있다. 소요 시간은 컴퓨터 한 대에 장착된 여러 GPU나 한걸음 더 나아가 여러 컴퓨터에 여러 GPU가 장착된 클러스터를 사용하는 방법으로 단축이 가능하다. 이런 구성은 다소 어려워 보이지만 다행히도 클라우드 서버 서비스를 사용하면 가능하다. DAWNBench(<https://dawn.cs.stanford.edu/benchmark/index.html>)는 스탠포드 대학에서 처음 만든 사이트로서, 공개 훈련 데이터셋으로 유명한 일반적인 딥러닝 작업을 수행했을 때의 훈련 시간과 클라우드 사용 비용을 벤치마크한 정보를 공개해뒀다.

아무튼 2부를 읽을 때쯤엔 GPU를 쓸 수 있다면 좋을 것이다. 아니라면 여러 클라우드 플랫폼 서비스에서 제공하는 내역을 점검해보자. 파이토치가 미리 설치되었고 GPU가 활성화된 주피터 노트북을 무제한 용량으로 제공하는 경우도 있다. 처음 시작할 경우라면 구글 코랩(<https://colab.research.google.com>)을 추천한다.

마지막으로 고려할 사항은 운영체제(이하 OS)다. 파이토치 최초 릴리스 버전은 리눅스와 맥OS에서 동작했고, 2018년에 이르러서야 윈도우를 지원했다. 현재 맥북은 CUDA를 지원하는 GPU가 없기 때문에 맥OS용 파이토치 패키지는 CPU만 사용 가능하다. 이 책 내용은 OS 유형에 상관없이 설명하지만 2부에 등장하는 일부 스크립트는 리눅스 배시 스크립트에서 돌아간다. 윈도우에서 돌리려면 살짝 변환을 해줘야 한다. 그 외 코드는 가능하면 편의상 주피터 노트에서 돌릴 수 있는 형태로 제공한다.

파이토치 설치 정보는 공식 웹사이트(<https://pytorch.org/get-started/locally>)의 시작하기^{Get Started} 가이드를 참고하자. 윈도우 사용자는 아나콘다(<https://www.anaconda.com/distribution>)와 미니콘다(<https://docs.conda.io/en/latest/miniconda.html>) 설치를 추천한다. 리눅스 운영체제의 설치 방법은 다양한데, 파이썬의 대표 패키지 관리 도구 pip로 할 수 있다. 의존 패키지 설치를 위해 requirements.txt 파일을 제공한다. 설치에 익숙하다면 원하는 개발환경과 호환되는 방식으로 자유롭게 설치해도 좋다.

2부에서 다루는 데이터를 내려받으려면 큰 저장 공간이 필요하고 다운로드 속도도 빨라야 한다. 중앙 탐지 프로젝트에서 사용하는 원본 데이터는 60GB 파일을 내려받아 압축을 풀면 120GB를 차지한다. 압축을 풀고 나면 내려받은 파일은 삭제하자. 훈련 과정에서 성능 개선을 위한 캐시 파일 용도로 80GB의 공간이 추가로 필요하기 때문에, 하드 디스크에는 최소 200GB의 여유 공간이 있어야 한다. 네트워크 스토리지를 사용할 수도 있지만 그 경우엔 로컬 디스크 보다는 훈련 속도가 매우 느릴 것이다. 빠른 데이터 저장 목적이라면 되도록 로컬 SSD에 공간을 확보하는 것이 좋다.

1.5.1 주피터 노트북

이제 파이토치를 비롯해 필요한 의존 패키지를 잘 설치했으며 모든 프로그램이 잘 동작하는 상태가 됐을 것이다. 책의 내용을 따라하는 방식에 대해 앞서 언급했는데, 우리는 전체 예제에서 주피터 노트북(Jupyter Notebook)을 많이 사용할 예정이다. 주피터 노트북은 브라우저에서 코드를 인터랙티브하게 수행할 수 있는 웹페이지를 제공한다. 작성한 코드는 커널(kernel)이 읽어들이는데 커널은 코드를 실행한 후 결과를 반환하는 서버에서 동작하는 프로세스로 보면 되며, 결과는 웹 페이지에 바로 나타난다. 노트북은 커널의 상태를 유지하기 때문에 코드 수행시 정의한 변수나 메모리 내용을 수행 종료나 재시작 전까지 유지한다. 주피터 노트북과 상호작용하는 기본 단위는 셀(cell)이다. 여기에 코드를 작성하고 메뉴에서 실행 버튼을 누르거나 시프트와 엔터 키를 동시에 누르면 커널이 셀 내 코드를 실행한다. 노트북에 여러 셀을 넣을 수 있고 새로운 셀은 이전 셀에서 만든 변수에 접근할 수 있다. 셀의 마지막 줄이 반환하는 값은 실행 후 셀 바로 아래에 출력되며 값 대신 그래프를 출력할 수도 있다. 소스 코드나 실행 결과 그리고 마크다운(Markdown) 형식의 텍스트 셀을 조화롭게 섞어서 멋진 인터랙티브 문서를 만들 수 있다. 주피터 노트북에 대한 모든 것은 프로젝트 웹사이트(<https://jupyter.org>)에 잘 나와 있다.

여기서, 깃허브에서 체크아웃 받은 코드의 루트 디렉토리에서 노트북 서버를 한번 실행해보자. 운영체제나 주피터 설치 환경에 따라 실행 모습이 달라보일 수 있다. 궁금한 점은 이 책의 포럼을 활용하기 바란다.⁶ 잘 동작한다면 기본 브라우저에 로컬 노트북 파일 리스트가 보일 것이다.

노트

주피터 노트북은 아이디어를 코드로 표현하고 확인할 수 있는 강력한 도구다. 우리 저자진은 이 책에서 다루는 사례에 최적의 도구라고 생각하지만 모두가 그렇게 느끼지는 않을 수도 있다. 상황에 따라 다르므로 마찰을 줄이고 인지적인 부하를 최소화하려면 파이토치 실험을 위해 자신이 원하는 방법이나 도구를 사용하기 바란다.

책에 나오는 모든 코드는 웹사이트(<https://www.manning.com/books/deep-learning-with-pytorch>)와 깃허브 저장소(<https://github.com/deep-learning-with-pytorch/dlwpt-code>)에서 볼 수 있다.

1.7 핵심 요약

- 딥러닝 모델은 예제를 학습해서 주어진 입력에 대해 기대하는 결과를 출력하는 방법을 자동으로 배운다.
- 파이토치 등의 라이브러리를 통해 우리는 효과적으로 신경망 모델을 만들고 훈련시킬 수 있다.
- 파이토치는 유연함과 속도에 집중하며 알아보기도 쉽다. 또한 연산을 위해 즉시 실행을 기본으로 지원한다.
- 토치스크립트를 활용하면 모델을 미리 컴파일해서 파이썬뿐 아니라 C++ 프로그램이나 모바일 기기에서도 모델을 동작시킬 수 있다.
- 2017년 초 파이토치 릴리스 후로 딥러닝 도구 생태계는 상당한 통합을 이뤄왔다.
- 파이토치는 딥러닝 프로젝트를 만드는 다양한 유틸리티 라이브러리를 제공한다.